

Agentic Scrum

The developer as Product Owner, AI agents as the team — a human-in-the-loop,
board-driven method for autonomous software delivery

Antonio Riccio · DomoticLab

Whitepaper v1 · July 8, 2026

Contents

Abstract	2
1. The problem	2
2. The method	2
3. The pipeline	2
4. Why it works	3
5. Benefits	3
6. Honest limitations	3
7. Conclusion	4
License and citation	5

Abstract

Coding agents can now write, test and ship real code. The open question is not *whether* they can code, but *how a human stays in control* without becoming a bottleneck. **Agentic Scrum** answers this by inverting the roles: the human acts purely as **Product Owner / Scrum Master**, and AI agents act as the **development team**. The interface between them is an ordinary **Jira board**, and the human-in-the-loop control is encoded in a single, visible signal — **the card’s assignee**. The result is a full software delivery pipeline (ideation, planning, development, review, release) that runs autonomously, yet never advances past a human decision without an explicit human hand-off.

The name places the method within terms already established in 2026 — *AI-Augmented Scrum*, *agentic coding / agentic SDLC*, *loop engineering* — while its distinctive contribution is the use of **the card’s assignee as the baton of human control**. Reference implementation: *Spoolio*, open source.

1. The problem

Two failure modes dominate “AI does the coding” setups:

1. **The human is a bottleneck** — every step needs a prompt, a copy-paste, a babysitting session. The agent is fast; the human is the throughput ceiling.
2. **The human loses control** — the agent runs free and merges questionable code, and nobody can tell *whose turn it is* or *what still needs a human*.

Agentic Scrum removes both by making control **structural** rather than manual.

2. The method

Roles. The developer never writes code. They do sprint planning, set priorities, review, and decide releases — the work of a PO/Scrum Master. Agents do ideation, planning, coding, and integration.

The board is the interface. Work lives on a Jira board with the usual columns (Bug, Idea, Todo, Develop, Validate, To Fix, To Release, Done, Blocked). The *status* says which stage a card is in.

The assignee is the baton. Orthogonal to status, the **assignee** says *whose turn it is right now*:

- assigned to the **human** — the automation ignores it (a human gate);
- assigned to the **bot** — the automation acts on it;
- kept assigned to the human — untouched, so a card can be driven manually.

Every hand-off is a re-assignment. This makes human-in-the-loop **explicit, auditable, and visible** — you can look at the board and instantly see what is waiting on you versus what the agents own.

3. The pipeline

- **Intake** — a GitHub issue becomes a *Bug* card, assigned to the human for triage.
- **Ideation** — an agent reads the codebase and proposes grounded feature ideas (each anchored to a real file), as *Idea* cards for the human to triage.
- **Planning** — for cards the human hands to the bot, an agent writes a precise technical plan as a comment, then hands it back. Feedback loops: the human comments, re-assigns, and the plan is *revised* to incorporate it.

- **Development** — an agent creates a `features/<card-id>` branch off `develop`, implements the plan, runs build/tests as a quality gate, pushes the branch, and moves the card to *Validate* for human review (or *Blocked* with a precise reason).
- **Release to develop** — a deterministic step opens and merges the feature PR into `develop`; the card goes to *Done*.
- **Release to production** — an on-demand step opens the `develop -> main` PR; **the human merges it** (they are the code owner) and the version is tagged. Production is never merged by the bot alone.

4. Why it works

- **Model tiering and cost gates.** Every scheduled step runs a *zero-LLM pre-check* first: if there is no work assigned to the bot, the model is never invoked — you pay only for real work. The model is configurable per stage: a cheap-but-capable model for coding, a premium model for planning, a free model for light tasks. The architecture is model-agnostic: the smarter the model, the better the output — with no change to the pipeline.
- **Security by encapsulation.** Agents never touch raw secrets. Credentials live in an env file / CI secrets and are exposed only through thin helper commands and a git credential helper, so a “chatty” model can never print a token.
- **Trunk safety (GitFlow).** `develop` is the integration branch (PR-only); `main` is production, gated by a code-owner review that is the human. External contributors fork and open PRs against `develop` — nobody pushes to `main` but the owner.
- **Self-hosted and portable.** The orchestrator runs on the developer’s own machine (or any server), talks to models via a provider router, and to Jira/GitHub via their APIs. No lock-in; the whole configuration is portable.

5. Benefits

- **Throughput decoupled from the human.** The human reviews decisions, not keystrokes.
- **Control that scales.** Human-in-the-loop is enforced by design, visible on the board, and safe by default.
- **Cost proportional to value.** Idle cycles cost nothing.
- **Future-proof.** As models improve, quality rises automatically; you swap a model ID, not a process.
- **Auditable.** Every action is attributed (bot vs human), every plan and decision lives on the card.

6. Honest limitations

- Code quality still tracks model capability — a weak model loops; a capable one delivers.
- Some platform limits are real (e.g. per-branch push restrictions differ between personal and organization repos) and are worked around, not ignored.
- Autonomous merges to the integration branch rely on a human having reviewed the code first; production merges stay manual by design.
- This is an engineering methodology, not an academic result: the claims here are architectural, not a benchmarked study. A natural next step is to measure throughput, cost per feature, and review time.

7. Conclusion

Agentic Scrum is a practical answer to “let the model write the code, keep the human in charge.” By turning a Jira board into the human-agent interface and the *assignee* into the control signal, the developer becomes the orchestrator of an AI team — steering *what* and *whether*, while agents handle *how*. It is available today, cheap to run, safe by construction, and it gets better on its own as models get smarter.

License and citation

Author: Antonio Riccio — DomoticLab

Contact: riccio.antonio19@gmail.com

Repository: <https://github.com/AntonioR199/Spoolio-opensource>

Version: Whitepaper v1 — July 8, 2026

License. This document is released under the Creative Commons Attribution 4.0 International license (CC BY 4.0). You are free to share and adapt it, including for commercial purposes, provided you give appropriate credit and indicate if changes were made. Full license text: <https://creativecommons.org/licenses/by/4.0/>

How to cite. Riccio, A. (2026). *Agentic Scrum: the developer as Product Owner, AI agents as the team*. DomoticLab. <https://github.com/AntonioR199/Spoolio-opensource>